32

Vector Databases

Full systems for storing and searching vectors

The production step up from FAISS

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A database built to store and search vectors
vs FAISS	A full system, not just a search library
Adds	Storage, live updates, filtering, scaling
Examples	Pinecone, Chroma, Weaviate, Qdrant
For	Production RAG apps and large knowledge bases

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**What is a Vector Database?**	Beyond a search library
2	**FAISS vs Vector Database**	Library vs full system
3	**What It Does**	Store, search, update, filter, scale
4	**Metadata Filtering**	Search within categories
5	**Popular Vector Databases**	The main options
6	**Using One in Code**	A Chroma example
7	**When to Use What**	FAISS or a vector DB?
8	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference

1. What is a Vector Database?

A vector database is a complete system built specially to store and search vectors. You learned FAISS, which does the fast searching. A vector database wraps that searching power inside a full database — one that also handles storing your data safely, updating it live, filtering it, and growing to huge sizes without you managing all the details yourself.

Think of it this way: FAISS is a brilliant search engine. A vector database is the whole library building — it has the search engine inside, plus shelves to store everything, librarians to add and remove books, a catalogue to filter by topic, and room to expand. For a real, live application, you usually want the whole building, not just the engine.

> ■ *This is the natural next step after FAISS. Your RAG chatbot used FAISS directly, which is great for a project. As apps grow into production, vector databases handle the extra real-world needs automatically.*

2. FAISS vs Vector Database

The key difference: FAISS is a library (a tool you call from your code), while a vector database is a service (a system that runs and manages everything for you).

Fig 1 — FAISS is a search library you manage. A vector database is a full system that manages everything.

	FAISS (library)	Vector Database
What it is	A search tool in your code	A full running system
Storage	You save/load files yourself	Handled automatically
Live updates	Awkward to add/delete	Easy, instant updates
Filtering	Not built in	Built-in metadata filters
Scaling	You manage it	Scales for you
Best for	Projects, prototypes	Production apps

3. What It Does

A vector database brings together everything a real app needs around vector search. Here are its core jobs.

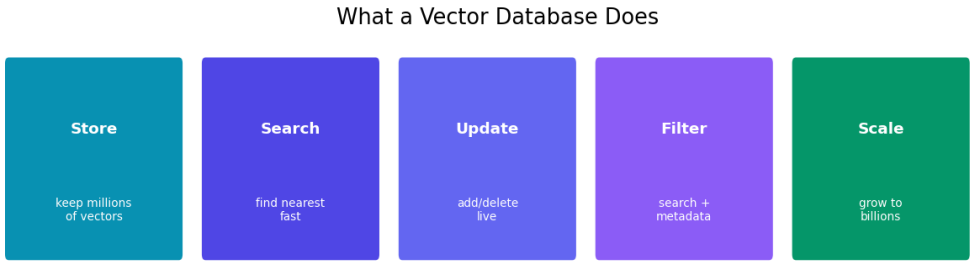

Fig 2 — The five core jobs of a vector database: store, search, update, filter, and scale.

Job	What it means
Store	Safely keep millions of vectors plus their text and info
Search	Find nearest vectors fast (the FAISS-style job)
Update	Add, change, or delete vectors live, without rebuilding
Filter	Combine vector search with rules (by date, category, user)
Scale	Grow to billions of vectors and many users smoothly

The big wins over plain FAISS are live updates and filtering. With FAISS, adding new documents often means rebuilding the index. A vector database lets you add and remove vectors any time, instantly — essential when your knowledge base changes.

4. Metadata Filtering

This is a standout feature. Metadata is extra info stored alongside each vector — like which document it came from, its date, category, or which user it belongs to. A vector database can search by meaning AND filter by metadata at the same time.

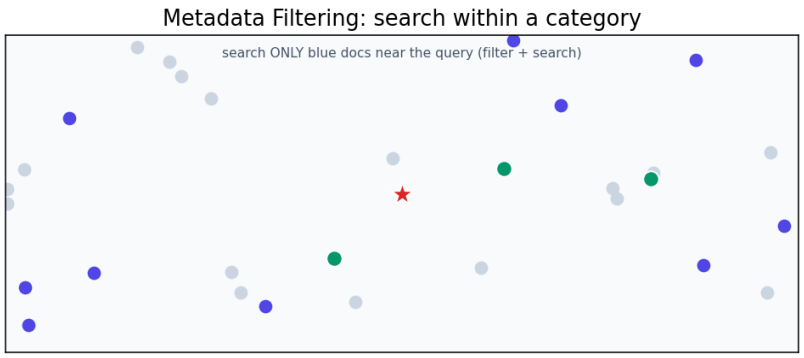

Fig 3 — Metadata filtering: search only within a chosen category (blue), ignoring the rest.

Example: 'find chunks similar to this question, but only from documents published in 2024, and only ones this user is allowed to see.' Plain FAISS can't do that filtering. A vector database does it in one query — combining semantic search with normal database-style rules. This is crucial for real apps with many users and document types.

5. Popular Vector Databases

There are several good options, each with a different flavour. You don't need to learn them all — just recognise the names and pick one that fits.

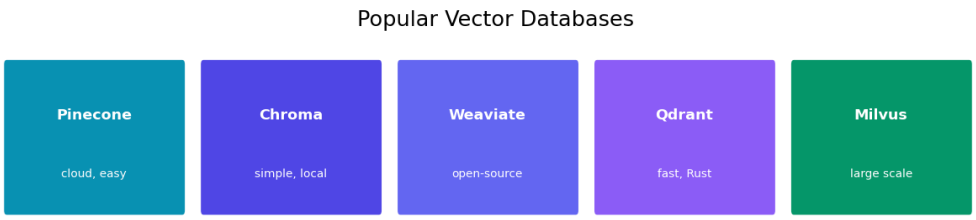

Fig 4 — Some of the most popular vector databases.

Database	Known for
Chroma	Simple, runs locally — great for learning and small apps
Pinecone	Fully managed cloud service — easy, no setup
Weaviate	Open-source, feature-rich
Qdrant	Fast, written in Rust, open-source
Milvus	Built for very large scale

■ *For learning and small projects, Chroma is the friendliest — it runs locally with almost no setup, much like using FAISS but with the extra database features. Pinecone is popular when you want a managed cloud service and don't want to run anything yourself.*

6. Using One in Code

Here's Chroma, a beginner-friendly vector database. Notice it handles embedding, storage, and search together — simpler than wiring up FAISS by hand.

```
# pip install chromadb
import chromadb

# Create a database and a collection
client = chromadb.Client()
collection = client.create_collection('my_docs')

# Add documents (Chroma embeds them for you)
collection.add(
    documents=['cats are pets', 'python is code', 'dogs are loyal'],
    metadatas=[{'topic': 'animal'}, {'topic': 'tech'}, {'topic': 'animal'}],
    ids=['1', '2', '3'])

# Search by meaning
results = collection.query(
    query_texts=['tell me about pets'],
    n_results=2)

# Search by meaning AND filter by metadata
results = collection.query(
    query_texts=['tell me about pets'],
    n_results=2,
    where={'topic': 'animal'})    # only animal docs

print(results['documents'])
```

■ *Notice the 'where' filter — that's metadata filtering in action. Chroma also stores everything to disk for you and lets you add or delete documents any time, no index rebuild needed.*

7. When to Use What

You don't always need a full vector database. Here's how to choose between plain FAISS and a vector database.

Use FAISS when...	Use a Vector DB when...
Building a project or prototype	Building a real production app
Data fits in memory, rarely changes	Data changes often (add/delete live)
You want full control / simplicity	You need filtering by metadata
No metadata filtering needed	Many users, large scale
Learning the concepts	You want storage handled for you

Practical path: learn and prototype with FAISS (like your RAG chatbot), then move to a vector database when you build something real that needs live updates, filtering, or scale. Chroma is an easy first step because it feels similar to FAISS but adds the database features. Many tutorials and tools (like LangChain, coming up) make swapping between them simple.

8. Common Mistakes

Mistake	Why it's a problem	Fix
Vector DB for a tiny project	Extra complexity for nothing	FAISS is enough for small/static data
Forgetting metadata	Can't filter later	Store useful metadata with vectors
Mismatched embed models	Search quality drops	Same model for adding and querying
Not using filters	Slower, less relevant	Filter by metadata when you can
Ignoring cost	Cloud DBs charge by usage	Check pricing before scaling
Picking the fanciest one	Overkill, harder to learn	Start simple (Chroma)

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
Vector database	full system to store + search vectors
vs FAISS	a service, not just a search library
Core jobs	store, search, update, filter, scale
Metadata	extra info stored with each vector
Metadata filter	search by meaning AND rules
Chroma	simple, local, beginner-friendly

Pinecone	managed cloud service
Install Chroma	pip install chromadb
Create collection	client.create_collection('name')
Add docs	collection.add(documents, ids)
Search	collection.query(query_texts, n_results)
Filter	query(..., where={'key': 'value'})

The one idea to remember: a vector database is FAISS-style search wrapped in a full system that also stores, updates, filters, and scales your vectors. Use FAISS for projects and prototypes; reach for a vector database when you build a real app that needs those production features.

Next Topic: RAG — Retrieval-Augmented Generation. We finally put all the pieces together: embeddings, vector search, and an LLM, into the complete system behind your PDF chatbot.